# 5.5. Bankruptcy in Taiwan TW

In [ ]:
```
# Import libraries here
```

# Prepare Data

## Import

**Task 5.5.1**

In [1]:
```
# Load data file
import gzip
import json

with gzip.open("data/taiwan-bankruptcy-data.json.gz", "rt") as f:
    taiwan_data = json.load(f)

# Check the type
print(type(taiwan_data))
```

```
<class 'dict'>
```

**Task 5.5.2**

> **Tip:** The data in this assignment might be organized differently than the data from the project, so be sure to inspect it first.

In [3]:
```
taiwan_data_keys = taiwan_data.keys()
print(taiwan_data_keys)
```

```
dict_keys(['schema', 'metadata', 'observations'])
```

In [4]:
```
# taiwan_data_keys = ...
# print(taiwan_data_keys)
```

**Task 5.5.3**

In [12]:
```
n_companies = len(taiwan_data["observations"])
print(n_companies)
```

```
6137
```

In [7]:
```
# n_companies = ...
# print(n_companies)
```

**Task 5.5.4**

In [14]:
```
n_features = len(taiwan_data["observations"][0])
print(n_features)
```

```
97
```

In [16]:
```
# n_features = ...
# print(n_features)
```

**Task 5.5.5**

In [17]:
```
# Create wrangle function

import gzip
import json
import pandas as pd

def wrangle(path):
    # Open and load the compressed JSON file
    with gzip.open(path, "rt") as f:
        data = json.load(f)

    # Extract observations
    observations = data["observations"]

    # Convert to DataFrame
    df = pd.DataFrame(observations)

    # Set id as index
    df = df.set_index("id")

    return df
```

In [18]:
```
df = wrangle("data/taiwan-bankruptcy-data.json.gz")

print("df shape:", df.shape)
df.head()
```

```
df shape: (6137, 96)
```

Out[18]:

| id | bankrupt | feat_1 | feat_2 | feat_3 | feat_4 | feat_5 | feat_6 | feat_7 | feat_ |
|---|---|---|---|---|---|---|---|---|---|
| 1 | True | 0.370594 | 0.424389 | 0.405750 | 0.601457 | 0.601457 | 0.998969 | 0.796887 | 0.80880 |
| 2 | True | 0.464291 | 0.538214 | 0.516730 | 0.610235 | 0.610235 | 0.998946 | 0.797380 | 0.80930 |
| 3 | True | 0.426071 | 0.499019 | 0.472295 | 0.601450 | 0.601364 | 0.998857 | 0.796403 | 0.80838 |
| 4 | True | 0.399844 | 0.451265 | 0.457733 | 0.583541 | 0.583541 | 0.998700 | 0.796967 | 0.80896 |
| 5 | True | 0.465022 | 0.538432 | 0.522298 | 0.598783 | 0.598783 | 0.998973 | 0.797366 | 0.80930 |

5 rows × 96 columns

# Explore

### Task 5.5.6

In [21]:
```
nans_by_col = df.isna().sum()

print("nans_by_col shape:", nans_by_col.shape)
nans_by_col.head()
```

```
nans_by_col shape: (96,)
```

Out[21]:
```
bankrupt    0
feat_1      0
feat_2      0
feat_3      0
feat_4      0
dtype: int64
```

In [20]:
```
# nans_by_col = ...
# print("nans_by_col shape:", nans_by_col.shape)
# nans_by_col.head()
```

**Slight Code Change**

In the following task, you'll notice a small change in how plots are created compared to what you saw in the lessons. While the lessons use the global matplotlib method like `plt.plot(...)`, in this task, you are expected to use the object-oriented (OOP) API instead. This means creating your plots using `fig, ax = plt.subplots()` and then calling plotting methods on the `ax` object, such as `ax.plot(...)`, `ax.hist(...)`, or `ax.scatter(...)`.

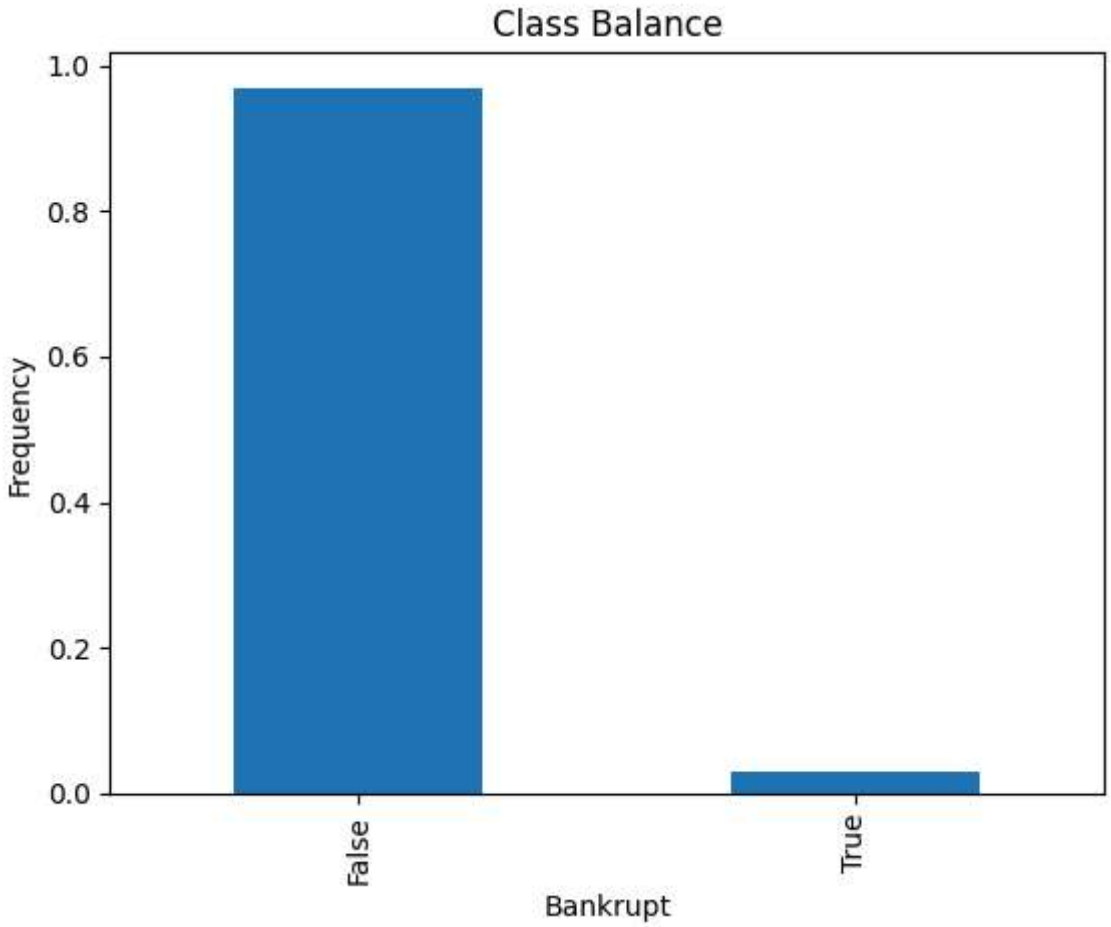
If you're using pandas' or seaborn's built-in plotting methods (like `df.plot()` or `sns.lineplot()`), make sure to pass the `ax=ax` argument so that the plot is rendered on the correct axes.

This approach is considered best practice and will be used consistently across all graded tasks that involve matplotlib.

### Task 5.5.7

In [23]:
```
# Plot class balance
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

# Plot normalized value counts
df["bankrupt"].value_counts(normalize=True).plot(kind="bar", ax=ax)

# Labels and title
ax.set_xlabel("Bankrupt")
```

```
ax.set_ylabel("Frequency")
ax.set_title("Class Balance")

plt.show()
```




# Split

### Task 5.5.8

In [25]:
```
target = "bankrupt"

# Feature matrix: all columns except the target
X = df.drop(target, axis=1)

# Target vector: only the target column
y = df[target]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (6137, 95)
y shape: (6137,)
```

In [ ]:
```
# target = ...
# X = ...
# y = ...
```

```
# print("X shape:", X.shape)
# print("y shape:", y.shape)
```

**Task 5.5.9**

In [27]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (4909, 95)
y_train shape: (4909,)
X_test shape: (1228, 95)
y_test shape: (1228,)
```

In [ ]:
```
# X_train, X_test, y_train, y_test = ...
# print("X_train shape:", X_train.shape)
# print("y_train shape:", y_train.shape)
# print("X_test shape:", X_test.shape)
# print("y_test shape:", y_test.shape)
```

# Resample

**Task 5.5.10**

In [29]:
```
from imblearn.over_sampling import RandomOverSampler

# Create the over sampler
over_sampler = RandomOverSampler(random_state=42)

# Apply over-sampling
X_train_over, y_train_over = over_sampler.fit_resample(X_train, y_train)

print("X_train_over shape:", X_train_over.shape)
X_train_over.head()
```

```
X_train_over shape: (9512, 95)
```

Out[29]:

| | feat_1 | feat_2 | feat_3 | feat_4 | feat_5 | feat_6 | feat_7 | feat_8 | feat_9 |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.535855 | 0.599160 | 0.594411 | 0.627099 | 0.627099 | 0.999220 | 0.797686 | 0.809591 | 0.303518 |
| **1** | 0.554136 | 0.612734 | 0.595000 | 0.607388 | 0.607388 | 0.999120 | 0.797614 | 0.809483 | 0.303600 |
| **2** | 0.549554 | 0.603467 | 0.599122 | 0.620166 | 0.620166 | 0.999119 | 0.797569 | 0.809470 | 0.303524 |
| **3** | 0.543801 | 0.603249 | 0.606992 | 0.622515 | 0.622515 | 0.999259 | 0.797728 | 0.809649 | 0.303510 |
| **4** | 0.498659 | 0.562364 | 0.546978 | 0.603670 | 0.603670 | 0.998904 | 0.797584 | 0.809459 | 0.304000 |

5 rows × 95 columns

In [ ]:
```
# over_sampler = ...
# X_train_over, y_train_over = ...
# print("X_train_over shape:", X_train_over.shape)
# X_train_over.head()
```

# Build Model

## Iterate

**Task 5.5.11**

In [31]:
```
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(random_state=42)
```

**Task 5.5.12**

> **Tip:** Use your CV scores to evaluate different classifiers. Choose the one that gives you the best scores.

In [33]:
```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(
    clf,
    X_train_over,
    y_train_over,
    cv=5,
    n_jobs=-1
)

print(cv_scores)
```

```
[0.99316868 0.99474514 0.99369085 0.99369085 0.9957939 ]
```

In [ ]:
```
# cv_scores = ...
# print(cv_scores)
```

In [35]:
```
# Run this code cell

params = {
    "max_depth": range(30, 50, 10),
    "n_estimators": range(25, 51, 25),
}
```

**Task 5.5.13**

In [36]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(random_state=42)

model = GridSearchCV(
    estimator=clf,
    param_grid=params,
    cv=5,
    n_jobs=-1,
    verbose=1
)
```

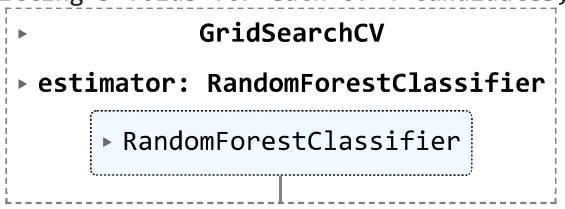
In [ ]:
```
# model = ...
```

**Ungraded Task:** Fit your model to the over-sampled training data.

In [38]:
```
model.fit(X_train_over, y_train_over)
```

```
Fitting 5 folds for each of 4 candidates, totalling 20 fits
```

Out[38]:




**Task 5.5.14**

In [39]:
```
import pandas as pd

cv_results = pd.DataFrame(model.cv_results_)
cv_results.head(5)
```

Out[39]:

| | mean_fit_time | std_fit_time | mean_score_time | std_score_time | param_max_depth | param_ |
|---|---|---|---|---|---|---|
| **0** | 0.733688 | 0.025048 | 0.008327 | 0.000359 | 30 | |
| **1** | 1.451721 | 0.021795 | 0.013882 | 0.001895 | 30 | |
| **2** | 0.731729 | 0.018852 | 0.008332 | 0.000323 | 40 | |
| **3** | 1.483151 | 0.032568 | 0.013575 | 0.000667 | 40 | |

In [ ]:
```
# cv_results = ...
# cv_results.head(5)
```

**Task 5.5.15**

In [41]:
```
best_params = model.best_params_
print(best_params)
```

```
{'max_depth': 40, 'n_estimators': 50}
```

In [ ]:
```
# best_params = ...
# print(best_params)
```

# Evaluate

**Ungraded Task:** Test the quality of your model by calculating accuracy scores for the training and test data.

In [43]:
```
acc_train = model.score(X_train_over, y_train_over)
acc_test = model.score(X_test, y_test)

print("Model Training Accuracy:", round(acc_train, 4))
print("Model Test Accuracy:", round(acc_test, 4))
```

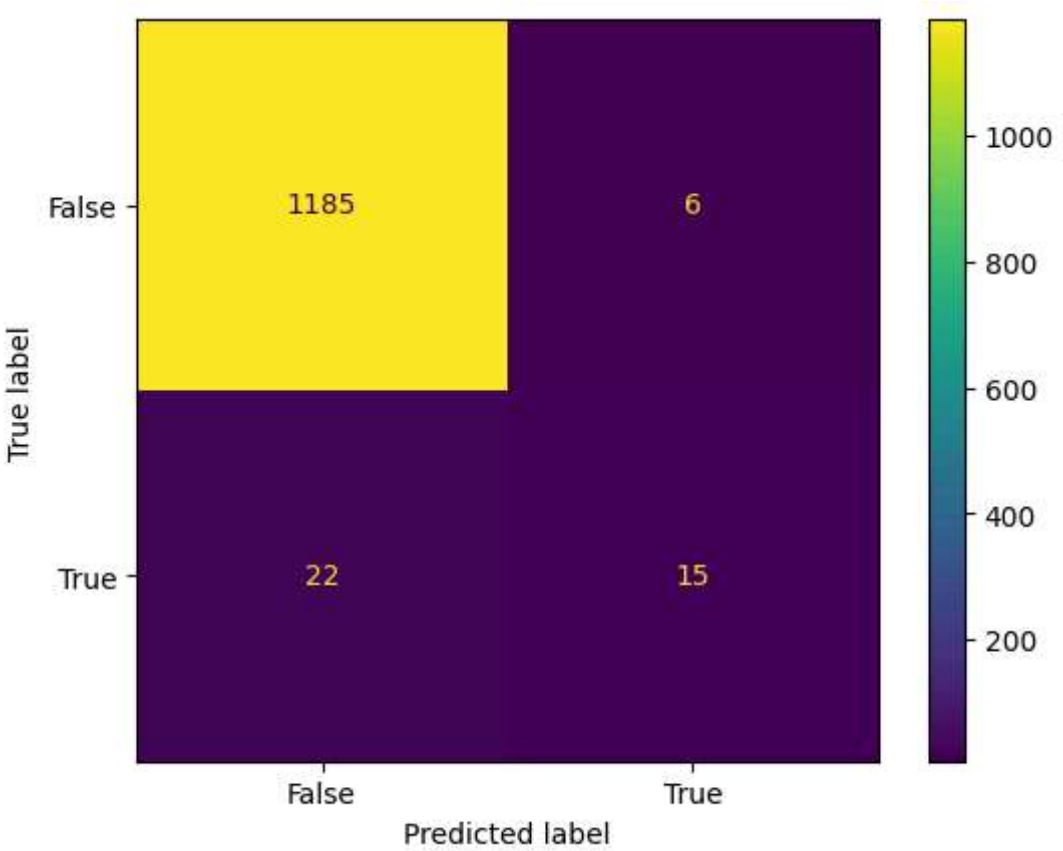
```
Model Training Accuracy: 1.0
Model Test Accuracy: 0.9772
```

In [ ]:
```
# acc_train = ...
# acc_test = ...
```

```
# print("Model Training Accuracy:", round(acc_train, 4))
# print("Model Test Accuracy:", round(acc_test, 4))
```

**Task 5.5.16**

In [48]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Predict on test set
y_pred = model.predict(X_test)

# Force correct label order
labels = [False, True]

cm = confusion_matrix(y_test, y_pred, labels=labels)

fig, ax = plt.subplots()
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(ax=ax)

plt.show()
```




**Task 5.5.17**

In [50]:
```
from sklearn.metrics import classification_report

# Generate classification report
class_report = classification_report(y_test, y_pred)
```

```
# Print it
print(class_report)
```

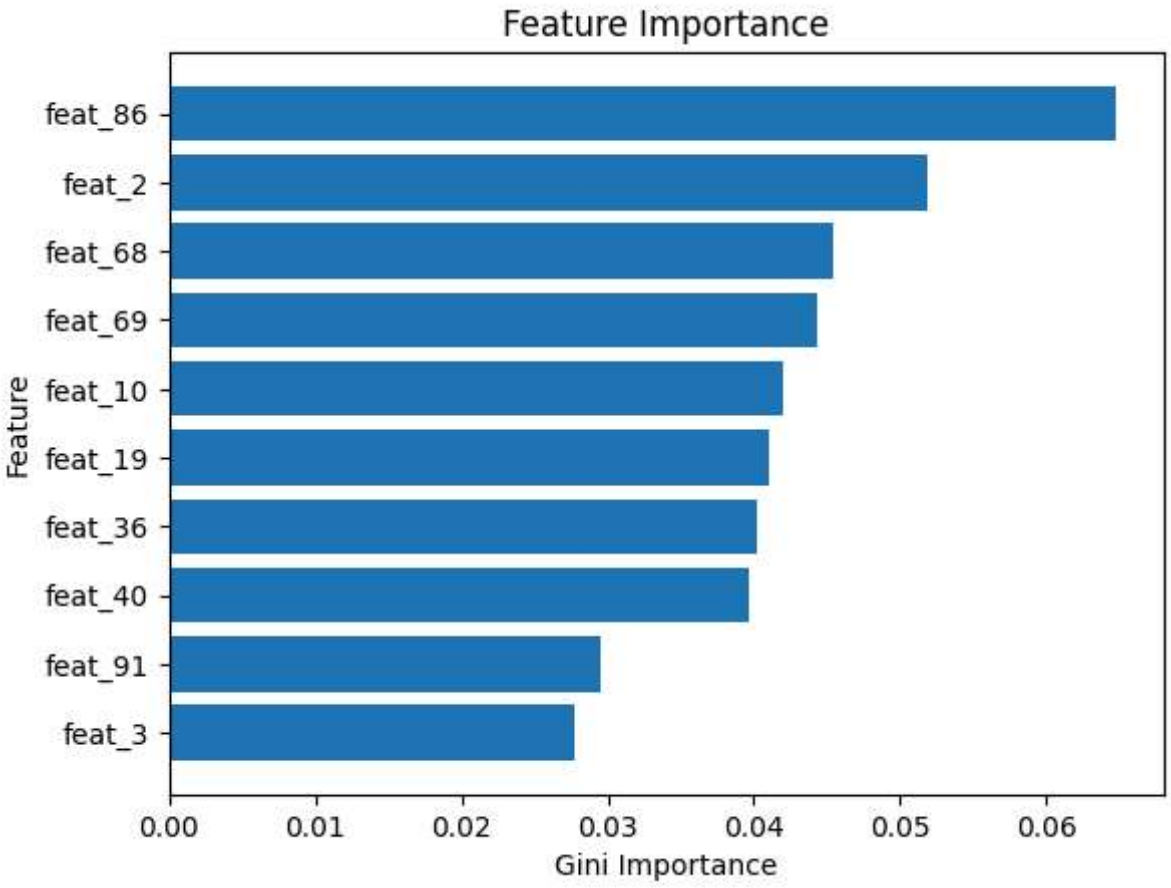
```
              precision    recall  f1-score   support

       False       0.98      0.99      0.99      1191
        True       0.71      0.41      0.52        37

    accuracy                           0.98      1228
   macro avg       0.85      0.70      0.75      1228
weighted avg       0.97      0.98      0.97      1228
```

In [ ]:
```
# class_report = ...
# print(class_report)
```

# Communicate

### Task 5.5.18

In [65]:
```
import matplotlib.pyplot as plt
import pandas as pd

# Ensure best model is refitted
best_model = model.best_estimator_

# Get feature importances
importances = best_model.feature_importances_

# Create dataframe
feat_importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Gini Importance': importances
})

# Sort and select top 10
top10 = feat_importance.sort_values(by='Gini Importance', ascending=False).head(10)

# Plot
fig, ax = plt.subplots()

# Reverse order for horizontal bar chart
ax.barh(top10['Feature'][::-1], top10['Gini Importance'][::-1])

ax.set_xlabel('Gini Importance')
ax.set_ylabel('Feature')
ax.set_title('Feature Importance')

plt.show()
```




**Task 5.5.19**

In [66]:
```
# Save model
import joblib

# Save the best model
joblib.dump(best_model, "model-5-5.pkl")
```

Out[66]:
```
['model-5-5.pkl']
```

**Task 5.5.20**

In [82]:
```
import pandas as pd
import joblib

def wrangle(df):
    """
    Add your preprocessing steps here.
    """
    # Example preprocessing:
    # df = df.drop(columns=["id"], errors="ignore")
    # df = df.fillna(0)

    return df


def make_predictions(data_filepath, model_filepath):
```

```
    """
    Load data, wrangle it, load model, and return predictions.
    """
    # Load test data
    df = pd.read_json(data_filepath, compression="gzip")

    # Apply wrangling
    df = wrangle(df)

    # Load the trained model
    model = joblib.load(model_filepath)

    # Predict
    y_pred = model.predict(df)

    return pd.Series(y_pred)
```

In [81]:
```
from my_predictor_assignment import make_predictions

y_test_pred = make_predictions(
    data_filepath="data/taiwan-bankruptcy-data-test-features.json.gz",
    model_filepath="model-5-5.pkl",
)

print("predictions shape:", y_test_pred.shape)
y_test_pred.head()
```

```
---------------------------------------------------------------------------
ImportError                               Traceback (most recent call last)
Cell In[81], line 1
----> 1 from my_predictor_assignment import make_predictions
      3 y_test_pred = make_predictions(
      4     data_filepath="data/taiwan-bankruptcy-data-test-features.json.gz",
      5     model_filepath="model-5-5.pkl",
      6 )
      8 print("predictions shape:", y_test_pred.shape)

ImportError: cannot import name 'make_predictions' from 'my_predictor_assignment' (/
app/project/my_predictor_assignment.py)
```

In [ ]:
```
# # Import your module


# # Generate predictions
# y_test_pred = make_predictions(
#     data_filepath="data/taiwan-bankruptcy-data-test-features.json.gz",
#     model_filepath="model-5-5.pkl",
# )

# print("predictions shape:", y_test_pred.shape)
# y_test_pred.head()
```

In [79]:
```
!cat /app/project/my_predictor_assignment.py
```

```
# Create your masterpiece :)
```

**Tip:** If you get an `ImportError` when you try to import `make_predictions` from `my_predictor_assignment`, try restarting your kernel. Go to the **Kernel** menu and click on **Restart Kernel and Clear All Outputs**. Then rerun just the cell above. ☝️

---